

CSCI3160 Design and Analysis of Algorithms (2025 Fall)

Week 1: the RAM Computation Model

Instructor: Xiao Liang¹

Department of Computer Science and Engineering
Chinese University of Hong Kong

¹These slides are primarily based on materials prepared by [Prof. Yufei Tao](#) (please refer to [Prof. Tao's version from 2024 Fall](#) for the original content). Some modifications have been made to better align with this year's teaching progress, incorporating student feedback, in-class interactions, and my own teaching style and research perspective.

This is **not** a programming course.

Main take-away message from this course

Computer science is a branch of mathematics with its art reflected in the beauty of **algorithms**.

- Programming knowledge is not necessary to study algorithms.

Many people believe that this branch holds the future of mankind.

In mathematics (and hence, computer science) everything—including every term and symbol—must be rigorous.

Computer science is a subject where we

- ① first define a **computation model**, which is a **simple** yet **accurate** abstraction of a computing machine;
- ② then slowly build up a theory in this model from scratch.

Is there a universally good model?

- No. We need different models for different purposes.

Some examples:

- Finite Automata: Used in regular expressions, compiler design, and more. They are clean, relatively easy to understand, yet powerful enough for real-world applications.
- Turing Machines: Fundamental to computational complexity theory. They (or at least the vanilla version of them) are not sensitive to polynomial differences in time or space cost.
- Boolean/Arithmetic Circuits: Commonly used in cryptography and electronic design automation (EDA). These represent a non-uniform computational model.

How to Choose a Model for the Study of Algorithms? What Are the Criteria?

- Mechanically Implementable:
 - This ensures our original goal—implementing algorithms to solve real-world problems.
 - It helps avoid logical paradoxes in the theory you established.
- Sufficiently General: The model should be general enough to capture all natural steps involved in solving problems strategically. Example: Finite automata are limited in power—they cannot recognize the language $L = \{a^n b^n \mid n \geq 0\}$ because they lack memory to “count” and match the number of a ’s and b ’s. This makes them unsuitable for many algorithmic problems that require more expressive computational models.
- Sensitive Enough: The model should be able to capture fine-grained differences in resource usage like time and space. Example: We often need to distinguish between algorithms with time complexities like $O(n^2)$ and $O(n \log(n))$, especially in performance-critical applications. Turing machines, while foundational, treat all polynomial-time algorithms as equivalent in complexity theory (due to polynomial-time reductions), making them too coarse for such distinctions.

Our Choice of the Computational Model

The Random Access Machine (RAM) model

A machine has a **memory** and a **CPU**.

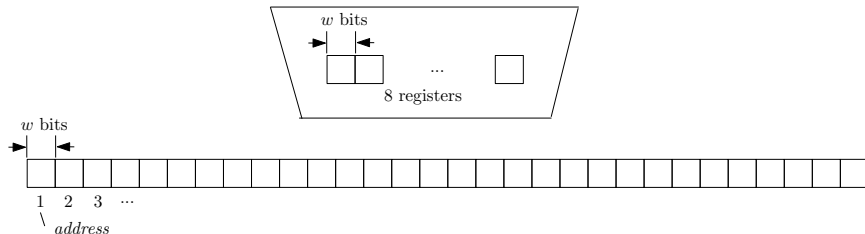
Memory

- An infinite **sequence** of **cells**, each of which contains the same number w of bits.
- Every cell has an **address**: the first cell of memory has address 1, the second cell 2, and so on.

The Random Access Machine (RAM) model

CPU

- Contains a fixed number—8 in this course—of **registers**, each of which has w bits (i.e., same as a memory cell).



The Random Access Machine (RAM) model

CPU

- Can do the following **atomic operations**:

1. **(Register (Re-)Initialization)**

Set a register to a fixed value (e.g., 0, -1 , 100, etc.), or to the content of another register.

The Random Access Machine (RAM) model

CPU

- Can do the following **atomic operations**:

2. (Arithmetic)

Take the integers a , b stored in two registers, calculate one of the following and store the result in a register:

- $a + b$, $a - b$, $a \cdot b$, and a/b .

Note: a/b is “integer division”, which returns an integer. For example, $6/3 = 2$ and $5/3 = 1$.

The Random Access Machine (RAM) model

CPU

- Can do the following **atomic operations**:

3. (Comparison/Branching)

Take the integers a, b stored in two registers, compare them, and learn which of the following is true:

- $a < b, a = b, a > b$.

The Random Access Machine (RAM) model

CPU

- Can do the following **atomic operations**:

4. (Memory Access)

Take a memory address A currently stored in a register. Do one of the following:

- Read the content of the memory cell with address A into a designated register (overwriting the bits there).
- Write the content of a designated register into the memory cell with address A (overwriting the bits there).

The Random Access Machine (RAM) model

CPU

- Can do the following **atomic operations**:

5. (Randomness)

- **RANDOM**(x, y): Given integers x and y (satisfying $x \leq y$), this operation returns an integer chosen **uniformly at random** in $[x, y]$, and places the random integer in a register.

The Random Access Machine (RAM) model

An **execution** is a sequence of atomic operations.

Its **cost** (also called its **running time**, or simply, **time**) is the **length** of the sequence, namely, the number of atomic operations.

The Random Access Machine (RAM) model

A **word** is a sequence of w bits, where w is called the **word length**.

- In other words, each memory cell and CPU register store a word.

Unless otherwise stated, you do not need to pay attention to the value of w in this course.

Algorithm

- An **input** refers to the initial state of the registers and the memory before an execution starts.
- An **algorithm** is a piece of description that, given an input, can be utilized to **unambiguously** produce a sequence of atomic operations, namely, the **execution** of the algorithm.
 - In other words, it should be always clear what the next atomic operation should be, given the outcome of all the previous atomic operations.
- The **cost** of an algorithm on an input is the length of its execution on that input (i.e., the number of atomic operations required).
- The **space** of an algorithm on an input is the **largest** memory address accessed by the algorithm's execution on that input.

Deterministic Algorithms vs. Random Algorithms

An algorithm is **deterministic** if it never invokes the atomic operation RANDOM. Otherwise, the algorithm is **randomized**.

On the same input, the cost of a deterministic algorithm is a fixed integer—it remains the same every time you execute the algorithm.

The cost of a randomized algorithm, however, is a **random variable**. Even on the same input, the cost may change each time the algorithm is executed.

Example

1. **do**
2. $r = \text{RANDOM}(0, 1)$
3. **until** $r = 1$

How many times would Line 2 be executed? The answer is—“we don’t know” (in fact, the line may be executed an infinite number of times)! Every time the above “algorithm” is executed, it may produce a new sequence of atomic operations.

Expected Cost of a Randomized Algorithm

Let X be a random variable that equals the cost of an algorithm on an input. The **expected cost** of the algorithm on the input is the expectation of X .

Wait a Moment: It is a well-known fact that **different distributions can share the same expected value, yet behave quite differently**. So,

- **Question:** Why is expected running time a good metric for analyzing randomized algorithms?

Why Expected Running Time?

Why is expected running time a good metric for analyzing randomized algorithms?

① Simplicity and Analytic Tractability:

- The expected running time is usually easier to compute and analyze than other probabilistic metrics (like the median, variance, or tail bounds).
- It provides a single, concrete number that summarizes the algorithm's typical performance.

② Linearity of Expectation:

- One of the most powerful tools in probability theory is the linearity of expectation. It allows us to analyze complex algorithms by breaking them into smaller parts and summing their expected costs—even if those parts are dependent.
- This makes expected running time a very modular and flexible tool in algorithm design.

③ Concentration Bounds: If the running time of a randomized algorithm has very high variance, the expectation may be misleading. In most cases, this can be reconciled if we utilize concentration bounds like

- the Markov inequality, Chebyshev inequality, or Chernoff bounds

Next, we will use Markov inequality as an example. (We also talked about Chernoff bounds during the first lecture. But that will not appear in quizzes/exams.)

Markov's Inequality

Markov's Inequality provides an upper bound on the probability that a non-negative random variable is much larger than its expectation.

Statement

Let X be a non-negative random variable and $a > 0$. Then:

$$\Pr[X \geq a] \leq \frac{\mathbb{E}[X]}{a}.$$

Intuition Behind Markov's Inequality

- If the average value of X is small, the chance that X is very large must be small.

Example

Suppose $\mathbb{E}[X] = 5$. Then:

$$\Pr[X \geq 25] \leq \frac{5}{25} = 0.2$$

So there's at most a 20% chance that X exceeds 25.

Applying Markov to Running Time

Let T be the random variable for the running time of a randomized algorithm.
Suppose:

- $T \geq 0$
- $\mathbb{E}[T] = \mu$

Then for any $c > 1$:

$$\Pr[T \geq c \cdot \mu] \leq \frac{1}{c}$$

Example

If $\mu = 10$, then:

$$\Pr[T \geq 100] \leq \frac{1}{10}$$

Example: Las Vegas Algorithm

- A Las Vegas algorithm always gives the correct result, with a random running time.
- Suppose $\mathbb{E}[T] = \mu$
- Run the algorithm with a timeout of $c \cdot \mu$

$$\Pr[\text{timeout}] \leq \frac{1}{c}$$

- Run for 3μ steps $\Rightarrow$ fail with probability $\leq \frac{1}{3}$
- Repeat 3 times $\Rightarrow$ succeed with good-enough probability $1 - \left(\frac{1}{3}\right)^3 \approx 0.963$.

CSCI3160 Design and Analysis of Algorithms (2025 Fall)

Measuring the Efficiency of an Algorithm by the Worst Input

Instructor: Xiao Liang¹

Department of Computer Science and Engineering
Chinese University of Hong Kong

¹These slides are primarily based on materials prepared by Prof. Yufei Tao (please refer to Prof. Tao's version from 2024 Fall for the original content). Some modifications have been made to better align with this year's teaching progress, incorporating student feedback, in-class interactions, and my own teaching style and research perspective.

A significant part of computer science is devoted to understanding the power of the RAM model in solving specific problems, that is, what would be a “fastest” algorithm for each problem.

But how do we measure “fast”? One approach—the one we follow in this course—is to look at the algorithm’s cost on the **worst** input, as we will formalize in this lecture.

Cost on the Worst Input

Define $\mathcal{I}_n$, where n is an integer, to be the set of all inputs to a problem that have the same **problem size** n .

Given an input $I \in \mathcal{I}_n$, the cost $X_{\mathcal{A}}(I)$ of an algorithm $\mathcal{A}$ is the length of its execution on I .

- The **worst-case cost** of $\mathcal{A}$ under the problem size n is the maximum $X_{\mathcal{A}}(I)$ of all $I \in \mathcal{I}_n$.
- The **worst expected cost** of $\mathcal{A}$ under the problem size n is the maximum $E[X_{\mathcal{A}}(I)]$ of all $I \in \mathcal{I}_n$.

Example: Dictionary Search

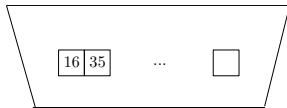
Problem Input: In the memory, a set S of n integers have been arranged in **ascending** order at the memory cells from address 1 to n . The value of n has been placed in Register 1 of the CPU. Another integer v has been placed in Register 2 of the CPU.

- n is the problem size.
- $\mathcal{I}_n$ is the set of all possible (S, v) .

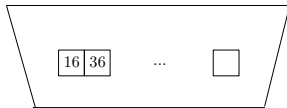
Goal: Determine **whether v exists in S .**

Example: Dictionary Search

A “yes”-input with $n = 16$

[illegible]

A “no”-input with $n = 16$

[illegible]

Example 1: Dictionary Search

The worst-case cost of the **binary search algorithm** is $O(\log n)$.

In other words, on any input in $\mathcal{I}_n$, the maximum number $f(n)$ of atomic operations performed by the algorithm grows no faster than $\log_2 n$.

Note: This does **not** mean $f(n) = \log_2 n$.

" $f(n) = O(\log n)$ " only says that $f(n)$ could be functions like $10(1 + \log_2 n)$, $352 \log_3 n$, $\sqrt{\log n} + 78 \log_2(n^{83})$, etc.

Example 2

Consider the following randomized algorithm:

/ A is an array of size n that contains at least one 0 */*

1. **do**
2. $r = \text{RANDOM}(1, n)$
3. **until** $A[r] = 0$
4. **return** r

What is the expected cost of the algorithm? The answer is “it depends”:

- If all numbers in A are 0, the algorithm finishes in $O(1)$ time.
- If A has only one 0, the algorithm finishes in $O(n)$ expected time because
 - $A[r]$ has $1/n$ probability of being 0.
 - In expectation, we need to repeat n times to find the 0. (Think: how to prove this claim formally?)

Example 2 (cont.)

/* A is an array of size n that contains at least one 0 */

1. **do**
2. $r = \text{RANDOM}(1, n)$
3. **until** $A[r] = 0$
4. **return** r

Worst-case cost of the algorithm = ∞

Worst expected cost of the algorithm = $O(n)$

We will finish the lecture by tapping into the power of randomization. We will see a problem where randomized algorithms are **provably faster** than deterministic ones in expected cost.

Before proceeding, think: what is the “expected cost” of a deterministic algorithm?

Power of Randomization

Problem “Find-a-Zero”: Let A be an array of n integers, among which half of them are 0. Design an algorithm to report an arbitrary position of A that contains a 0.

For example, suppose $A = (9, 18, 0, 0, 15, 0, 33, 0)$. An algorithm can report 3, 4, 6, or 8.

The Randomized Complexity of “Find-a-Zero”

Power of Randomization

1. **do**
2. $r = \text{RANDOM}(1, n)$
3. **until** $A[r] = 0$
4. **return** r

The algorithm finishes in $O(1)$ expected time on **every input** A !

Think: how to proof this claim formally?

The Deterministic Complexity of “Find-a-Zero”

In contrast, **any** deterministic algorithm must probe at least $n/2$ integers of A in the worst case!

Here are two caveats:

- Pay attention to the order of quantifiers: $\exists$ algorithm such that $\forall A \dots$
- Think: how to prove this claim formally? We need to do the following argument: we can treat any deterministic algorithm as making **black-box** queries to the array A , interleaved by some *deterministic* “local” computation steps. So, for any algorithm, you can always construct a “hard” A to enforce a worst-case performance for the given algorithm. (We presented the detailed derivation on the whiteboard. This isn't required for quiz/exam.)

Also note that: this proof relies crucially on the order of quantifiers!

In other words, any deterministic algorithm must have a worst case time of $\Theta(n)$ —provably slower than the above randomized algorithm ($O(1)$ in expectation).

CSCI3160 Design and Analysis of Algorithms (2025 Fall)

Basic Techniques: Recursion, Repeating, and Geometric Series

Instructor: Xiao Liang¹

Department of Computer Science and Engineering
Chinese University of Hong Kong

¹These slides are primarily based on materials prepared by [Prof. Yufei Tao](#) (please refer to [Prof. Tao's version from 2024 Fall](#) for the original content). Some modifications have been made to better align with this year's teaching progress, incorporating student feedback, in-class interactions, and my own teaching style and research perspective.

Today we will discuss three basic techniques of algorithm design:

- Recursion
- Geometric Series.
- Repeating (till success)

We will use two famous problems to demonstrate the ideas behind these techniques:

- Hanoi Tower: for recursion
- k -selection: for geometric series and repeating till success.

Recursion

Principle of recursion

When dealing with a subproblem (same problem but with a smaller input), consider it solved, and use the subproblem's output to continue the algorithm design.

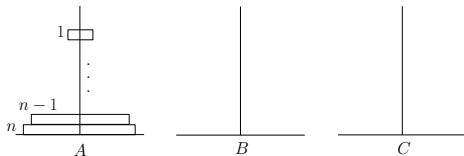
The Hanoi Tower Problem

Hanoi Tower

There are 3 rods A, B, and C.

On rod A, n disks of different sizes are stacked in such a way that no disk of a larger size is above a disk of a smaller size.

The other two rods are empty.

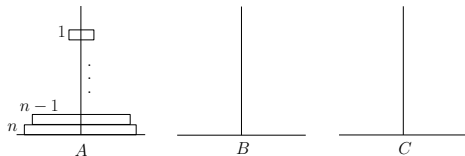


The Hanoi Tower Problem

Hanoi Tower

Permitted operation: Move the top-most disk of a rod to another rod.

Constraint: No disk of a larger size can be above a disk of a smaller size.



Goal: Design an algorithm to move all the disks to rod B.

On the Difficulty of the Hanoi Tower Problem

After some initial attempts, you will realize how hard this problem is.

Seems it is very challenging to design an *efficient* algorithm for that.

In this case, an important way of thinking in algorithm design (and complexity theory) is to ask:

- Is there any *inherent* difficulty in solving the concerned problem?

Such a difficulty, if exists, is usually referred to as a “lower bound”, “infeasibility”, or “impossibility” result. It is an important branch of research for both algorithm design and computational complexity.

Some remarks:

- This question is so natural. But people usually forget to ask when they really get stuck with some hard problems.
- Don't insist on proving impossibility results *unless you have some intuition to start with*.

Intuition on the Difficulty

Our intuition on the inherent difficulty of the Hanoi Tower Problem:

- Due to the constraint, we have to move the last disk to rob B , before we do that for other disks.
- However, to move the last disk, we have to move all other disks to rob C so we can start to move the last disk
- This problem exhibits a flavor of symmetry among the robs. And if we ignore the last disk, it is a new problem of $n - 1$ disks. So, the last item essentially asks to solve the exactly same problem but with one less disk.

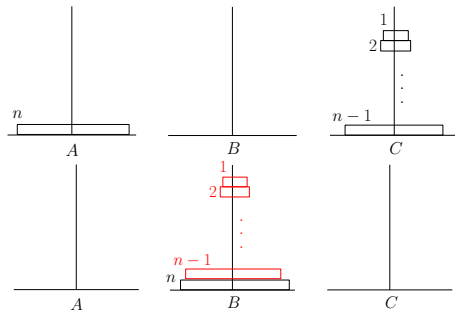
Importantly, the above intuition is *general*, i.e., it always applies no matter what algorithm you want to design.

Illustrating the Intuition

Subproblem: Same problem but with $n - 1$ disks.

Consider the subproblem solved (i.e., assume you already have an algorithm for it).

Now, solve the problem with n disks as follows:



Formalizing our Intuition

Suppose that our algorithm performs $f(n)$ operations to solve a problem of size n . Clearly, $f(1) = 1$. By recursion, we can write

$$f(n) \geq 1 + 2 \cdot f(n-1)$$

Solving this recurrence gives $f(n) \geq 2^n - 1$.

Conclusion: The best time complexity (even for randomized algorithms) for solving the Tower of Hanoi problem with n disks is $\Omega(2^n)$.

Geometric Series and Repeating till Success

The k -Selection Problem

The k -Selection Problem: You are given a set S of n integers in an (possibly unsorted) array and an integer $k \in [1, n]$. Find the k -th smallest integer of S .

For example, suppose that $S = (53, 92, 85, 23, 35, 12, 68, 74)$ and $k = 3$. You should output 35.

Definition of Rank

We will introduce a definition to assist our subsequent discussion:

The **rank** of an integer $v \in S$ is the number of elements in S smaller than or equal to v .

For example, suppose that $S = (53, 92, 85, 23, 35, 12, 68, 74)$. Then, the rank of 53 is 4, and that of 12 is 1.

Easy: The rank of v can be obtained in $O(|S|)$ time.

Consider the following task:

Task: Assume n to be a multiple of 3. Obtain a subproblem of size at most $2n/3$ with exactly the same result as the original problem.

Our goal is to produce a set S' and an integer k' such that

- $|S'| \leq 2n/3$
- $k' \in [1, |S'|]$
- The element with rank k' in S' is the element with rank k in S .

Note: it is possible that $k' \neq k$.

We will give an algorithm to accomplish the task in $O(n)$ expected time.

Next, we will focus on the following two claims:

- ① There exists a (randomized) algorithm A_{sub} that accomplish the previous task in $O(n)$ expected time. (This step utilizes Repeating till Success.)
- ② We can utilize A_{sub} to design a (randomized) algorithms that solves k -selection in $O(n)$ expected time. (This step utilizes Geometric Series.)

We will first see how the second item works. After that, we will present the algorithm A_{sub} .

Utilizing A_{sub}

- $A_{sub}(S, k) \rightarrow (S_1, k_1)$. Note that $|S_1| = \frac{2}{3} \cdot n$.
- $A_{sub}(S_1, k_1) \rightarrow (S_2, k_2)$. Note that $|S_2| = (\frac{2}{3})^2 \cdot n$.
- $A_{sub}(S_2, k_2) \rightarrow (S_3, k_3)$. Note that $|S_3| = (\frac{2}{3})^3 \cdot n$.
- ...

Stop until the t -th repetition such that $|S_t| = 1$, i.e.,

$$\left(\frac{2}{3}\right)^t = \frac{1}{n}.$$

You can of course solve for t from the above equation to calculate the running time of the algorithm. But we will do it in an alternative way, utilizing the Geometric Series.

Geometric Series

A **geometric sequence** is an infinite sequence of the form

$$n, cn, c^2n, c^3n, \dots$$

where n is a positive number and c is a constant satisfying $0 < c < 1$.

Also recall the formula for the sum of finite geometric series:

$$S_t = n \cdot \frac{1 - c^t}{1 - c}$$

It holds in general that

$$\sum_{t=0}^{\infty} c^t n = \lim_{t \rightarrow \infty} S_t = \lim_{t \rightarrow \infty} n \cdot \frac{1 - c^t}{1 - c} = \frac{n}{1 - c} = O(n).$$

The summation $\sum_{t=0}^{\infty} c^t n$ is called a **geometric series**.

Geometric series are extremely important for algorithm design.

Algorithm

Using the repeating technique, now you should be able to convert the problem to a subproblem with size at most $\lceil 2n/3 \rceil$ in $O(n)$ expected time. (Note: we have not shown that A_{sub} runs in $O(n)$ expected time yet. We will do that toward the end.)

Now, apply the recursion technique. We have already obtained a (complete) algorithm solving the k -selection problem!

Think: How is this related to geometric series?

Running Time Analysis via Geometric Series

Assume: the running time of A_{sub} is linear, i.e., $O(n)$. (We will prove this later.)

The expected running time of our eventual algorithm (which invokes A_{sub} until $|S_t| = 1$) will be (note that we also use the linearity of the expectation operator here)

$$a \cdot n + a \cdot \frac{2}{3} \cdot n + a \cdot \left(\frac{2}{3}\right)^2 \cdot n + \dots + a \cdot \left(\frac{2}{3}\right)^t \cdot n$$

where a is the constant hidden in the big- O notation (here we need to make it explicit). This is no larger than

$$\begin{aligned} & a \cdot n + a \cdot \frac{2}{3} \cdot n + a \cdot \left(\frac{2}{3}\right)^2 \cdot n + \dots + \dots \\ &= a \cdot n + a \cdot \sum_{i=1}^{\infty} \left(\frac{2}{3}\right)^i \cdot n \\ &= a \cdot n + a \cdot O(n) \quad (\text{by Geometric Series}) \\ &= O(n) \end{aligned}$$

Repeat till Success

Algorithm for the Subproblem

The algorithm A_{sub} :

- ① Take an element $v \in S$ uniformly at random.
- ② Divide S into S_1 and S_2 where
 - S_1 = the set of elements in S less than or equal to v ;
 - S_2 = the set of elements in S greater than v .
- ③ If $|S_1| \geq k$, then return $S' = S_1$ and $k' = k$;
else return $S' = S_2$ and $k' = k - |S_1|$.

The algorithm **succeeds** if $|S'| \leq 2n/3$, or **fails** otherwise.

Repeat the algorithm until it succeeds.

Algorithm for the Subproblem

Lemma: The algorithm succeeds with probability at least $1/3$.

Proof: The algorithm always succeeds when the rank of v falls in $[\frac{n}{3}, \frac{2}{3}n]$ (think: why?). This happens with a probability at least $1/3$, by the fact that v is taken from S uniformly at random. □

In general, if an algorithm succeeds with a probability **at least** $c > 0$, then the number of repeats needed for the algorithm to succeed for the first time is **at most** $1/c$ in expectation.

Running time of A_{sub}

The algorithm A_{sub} :

- ① Take an element $v \in S$ uniformly at random. ($O(1)$ in our model. Why?)
- ② Divide S into S_1 and S_2 where
 - S_1 = the set of elements in S less than or equal to v ;
 - S_2 = the set of elements in S greater than v .($O(n)$ in our model.)
- ③ If $|S_1| \geq k$, then return $S' = S_1$ and $k' = k$;
else return $S' = S_2$ and $k' = k - |S_1|$.
($O(n)$ in our model.)

Thus:

- Each execution cost $O(n)$ (deterministic) time.
- By the previous lemma, we need to repeat it for 3 times *in expectation* until it succeeds.
- This implies that the expected running time is $O(n)$ (think: why? - Linearity of expectation).

Closing Remark (1/2)

It may seem almost magical that we can suddenly define the algorithm A_{sub} , and it just happens to work so well. At first glance, it feels as though the solution appears out of thin air, without any clear path leading to its discovery.

In reality, the techniques we've introduced so far seems to primarily serve as tools for analysis, rather than *direct aids in constructing the algorithm itself*. They help us understand why the algorithm works, prove its correctness, and analyze its efficiency—but they don't necessarily guide us toward finding the algorithm in the first place.

Closing Remark (2/2)

This phenomenon is actually common in algorithm design. For many non-trivial algorithms, there is often a component that appears almost mysterious or unmotivated at first. These seemingly magical insights are not accidents — they are the product of scientific creativity, intuition, and experience.

In fact, this creative leap is what distinguishes routine problem-solving from true algorithmic innovation. The “magic” is often the result of deep understanding, abstraction, and pattern recognition developed over time. Once the core idea is discovered, the rest of the work—rigorous analysis, optimization, and formal proof—can follow in a more systematic way.

So while it may feel surprising that A_{sub} works so well, this is a reminder of a deeper truth: in algorithm design, insight often precedes explanation.

CSCI3160 Design and Analysis of Algorithms (2025 Fall)

Divide and Conquer

Instructor: Xiao Liang¹

Department of Computer Science and Engineering
Chinese University of Hong Kong

¹These slides are primarily based on materials prepared by [Prof. Yufei Tao](#) (please refer to [Prof. Tao's version from 2024 Fall](#) for the original content). Some modifications have been made to better align with this year's teaching progress, incorporating student feedback, in-class interactions, and my own teaching style and research perspective.

In this lecture, we will discuss the **divide and conquer** technique for designing algorithms with strong performance guarantees. Our discussion will be based on the following problems:

- ① Sorting (a review of merge sort)
- ② Counting inversions
- ③ Dominance counting
- ④ Matrix multiplication

Recall: Principle of recursion

When dealing with a subproblem (same problem but with a smaller input), consider it solved, and use the subproblem's output to continue the algorithm design.

- When dividing, we utilize recursion to reduce the original problem into subproblems.
- When conquering, we tackle the **core problem** “hidden within” the original problem.

Sorting

Sorting

Problem: Given an array A of n distinct integers, produce another array where the same integers have been arranged in ascending order.

The Merge Sort Algorithm:

- **Divide:** Let A_1 be the array containing the first $\lceil n/2 \rceil$ elements of A , and A_2 be the array containing the other elements of A .
Sort A_1 and A_2 recursively.
- **Conquer:** Merge the two sorted arrays A_1 and A_2 in ascending order. This can be done in $O(n)$ time.

Sorting

Running Time: Let $f(n)$ denote the worst-case cost of the algorithm on an array of size n . Then:

$$f(n) \leq 2 \cdot f(\lceil n/2 \rceil) + O(n)$$

which gives $f(n) = O(n \log n)$.

Counting Inversions

Counting Inversions

Let: A = an array of n distinct integers.

An **inversion** is a pair of (i, j) such that

- $1 \leq i < j \leq n$, and
- $A[i] > A[j]$.

Example: Consider $A = (10, 3, 9, 8, 2, 5, 4, 1, 7, 6)$.

Then $(1, 2)$ is an inversion because $A[1] = 10 > A[2] = 3$. So are $(1, 3)$, $(3, 4)$, $(4, 5)$, and so on. There are in total 29 inversions.

Think: Can you come up with a naive algorithm that solves this problem in time $O(n^2)$?

Answer: Compare for each pair one by one in order. It takes

$$(n-1) + (n-2) + \dots + 1 = O(n^2)$$

Counting Inversions

Problem: Given an array A of n distinct integers, count the number of inversions.

We will do in the class: $O(n \log^2 n)$ time.

You will do as an exercise: $O(n \log n)$ time.

Counting Inversions

- **Divide:** Let A_1 the array containing the first $\lceil n/2 \rceil$ elements of A , and A_2 be the array containing the other elements of A .

Solve the “counting inversions” problem recursively on A_1 and A_2 , respectively. By doing so, we have already obtained the number m_1 of inversions in A_1 , and similarly, the number m_2 for A_2 .

- **Conquer:**

It remains to count the number of **crossing inversions** (i, j) where $i \in A_1$ and $j \in A_2$.

Counting Inversions

A_1 = the array containing the first $\lceil n/2 \rceil$ elements of A

A_2 = the array containing the other elements of A .

Next, perform the following two steps:

- 1 Sort A_1 .
 - in $O(n \log n)$ using merge sort.
- 2 For each element $e \in A_2$, count how many crossing inversions e produces using **binary search**.
 - Since $|A_2| = \frac{n}{2}$, there are $n/2$ binary searches performed in total, which takes $O(n \log n)$ time.

Illustration by Example

Example (cont.): $A = (10, 3, 9, 8, 2, 5, 4, 1, 7, 6)$.

$A_1 = (2, 3, 8, 9, 10)$ (sorted), $A_2 = (5, 4, 1, 7, 6)$

Element 5 produces 3 crossing inversion

Element 4 produces 3, too.

Elements 1, 7, and 6 produce 5, 3, and 3 crossing inversions, respectively.

- **Think:** How to obtain each count with binary search?

Counting Inversions

Running Time: Let $f(n)$ denote the worst-case cost of the algorithm on an array of size n . Then:

$$f(n) \leq 2 \cdot f(\lceil n/2 \rceil) + O(n \log n)$$

which gives $f(n) = O(n \log^2 n)$.

Dominance Counting

Dominance Counting

Denote by $\mathbb{Z}$ the set of integers.

Given a point p in two-dimensional space $\mathbb{Z}^2$, denote by $p[1]$ and $p[2]$ its x- and y-coordinate, respectively.

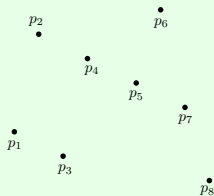
Given two distinct points p and q , we say that q **dominates** p if $p[1] \leq q[1]$ and $p[2] \leq q[2]$; see the figure below:



Dominance Counting

Let P be a set of n points in $\mathbb{Z}^2$ with distinct x-coordinates. Find, for **each** point $p \in P$, the number of points in P that are dominated by p .

Example:



We should output: $(p_1, 0), (p_2, 1), (p_3, 0), (p_4, 2), (p_5, 2), (p_6, 5), (p_7, 2), (p_8, 0)$.

Dominance Counting

Let P be a set of n points in $\mathbb{Z}^2$ with distinct x-coordinates. Find, for each point $p \in P$, the number of points in P that are dominated by p .

We will do in the class: $O(n \log^2 n)$ time.

You will do as an exercise: $O(n \log n)$ time.

Note: We can assume without loss of generality that the points are given in ascending order in their x-coordinates, i.e.,

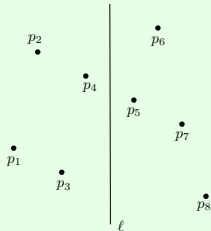
$$p_1[1] < p_2[1] < \dots < p_n[1].$$

This is without loss of generality because we can always sort them to be so in $O(n \log n)$ time, and our dominance counting algorithm anyway won't be faster than that ($O(n \log^2 n)$ or $O(n \log n)$).

Dominance Counting

Divide: Find a vertical line ℓ such that P has $\lceil n/2 \rceil$ points on each side of the line.

Example:



Think: How to find such ℓ in $O(n \log n)$ time? How about $O(n)$ time?

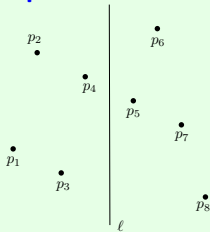
Dominance Counting

Divide:

P_1 = the set of points of P on the left of ℓ

P_2 = the set of points of P on the right of ℓ

Example:



$$P_1 = \{p_1, p_2, p_3, p_4\}$$

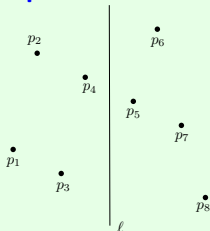
$$P_2 = \{p_5, p_6, p_7, p_8\}.$$

Dominance Counting

Divide:

Solve the dominance counting problem on P_1 and P_2 separately.

Example:



On P_1 , we have obtained:
 $(p_1, 0), (p_2, 1), (p_3, 0), (p_4, 2)$.

On P_2 , we have obtained: $(p_5, 0), (p_6, 1),$
 $(p_7, 0), (p_8, 0)$.

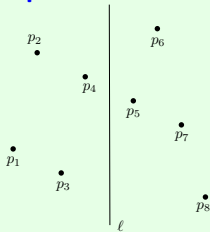
The counts obtained for the points in P_1 are final (**think:** why?).

Dominance Counting

Conquer:

It remains to count, for each point $p_2 \in P_2$, how many points in P_1 it dominates.

Example:



On P_2 , we have obtained: $(p_5, 0), (p_6, 1), (p_7, 0), (p_8, 0)$.

Regarding p_5 , for example, we still need to find out that it dominates 2 points from P_1 .

The x-coordinates do not matter any more!

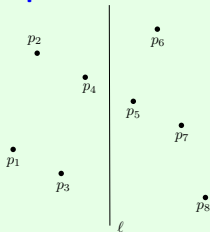
Dominance Counting

Conquer:

Sort P_1 by **y-coordinate**.

Then, for each point $p_2 \in P_2$, we can obtain the number points in P_1 dominated by p_2 using binary search.

Example:



P_1 in ascending of y-coordinate: p_3, p_1, p_4, p_2 .

How to perform binary search to obtain the fact that p_5 dominates 2 points in P_1 ?

- Search using the y-coordinate of p_5 .

Dominance Counting

Analysis:

Let $f(n)$ be the worst-case running time of the algorithm on n points. Then:

$$f(n) \leq 2f(\lceil n/2 \rceil) + O(n \log n)$$

which solves to $f(n) = O(n \log^2 n)$.

Matrix Multiplication

Matrix Multiplication

Problem: Given two $n \times n$ matrices A and B , compute their product AB .

We store an $n \times n$ matrix with an array of length n^2 in “row-major” order.

Example: $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ is stored as $(1, 2, 3, 4)$.

Note that any $A[i, j]$ — the element of A at the i -th row and j -th column — can be accessed in $O(1)$ time.

Trivial: $O(n^3)$ time

We will do in the class: $O(n^{2.81})$ time for n being a power of 2

You will do as an exercise: $O(n^{2.81})$ time for any n .

Matrix Multiplication

Warm Up: Suppose we want to compute $\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix}$. How many multiplication operations do we need to perform?

Trivial: 8.

Non-trivial: 7.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} p_5 + p_4 - p_2 + p_6 & p_1 + p_2 \\ p_3 + p_4 & p_1 + p_5 - p_3 - p_7 \end{bmatrix}$$

where

$$p_1 = a(f - h)$$

$$p_2 = (a + b)h$$

$$p_3 = (c + d)e$$

$$p_4 = d(g - e)$$

$$p_5 = (a + d)(e + h)$$

$$p_6 = (b - d)(g + h)$$

$$p_7 = (a - c)(e + f)$$

Matrix Multiplication (Strassen's Algorithm)

Recall that the input A and B are order- n (i.e., $n \times n$) matrices. Assume for simplicity that n is a power of 2. Divide each of A and B into 4 submatrices of order $n/2$:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

It is easy to verify:

$$AB = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}$$

How many order- $(n/2)$ matrix multiplications do we need?

Trivial: 8.

Non-trivial: 7 — see the next slide.

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} p_5 + p_4 - p_2 + p_6 & p_1 + p_2 \\ p_3 + p_4 & p_1 + p_5 - p_3 - p_7 \end{bmatrix}$$

$$p_1 = A_{11}(B_{12} - B_{22})$$

$$p_2 = (A_{11} + A_{12})B_{22}$$

$$p_3 = (A_{21} + A_{22})B_{11}$$

$$p_4 = A_{22}(B_{21} - B_{11})$$

$$p_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$p_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$p_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$

If $f(n)$ is the worst-case time of computing the product of two order- n matrices, then each of p_i ($1 \leq i \leq 7$) can be computed in $f(n/2) + O(n^2)$ time, where

- $f(n/2)$: each p_i contains exactly one multiplication between two order- $\frac{n}{2}$ matrices.
- $O(n^2)$: each p_i contains a constant number of addition/deduction between two order- $\frac{n}{2}$ matrices.

Matrix Multiplication

Therefore:

$$f(n) \leq 7f(n/2) + O(n^2)$$

which solves to $f(n) = O(n^{\log_2 7}) = O(n^{2.81})$.

Note: This can be derived using the Master Theorem. If you need a refresher, please review it. See Sections 4.2–4.5 of [CLRS].

Who Does the Heavy Lifting in Divide and Conquer?

Some students asked about certain seemingly odd aspects of divide-and-conquer (and recursive) algorithms. The question can be summarized as follows:

- The intermediate steps in these algorithms often feel “hollow,” making it unclear which part of the implementation is doing the real work.

This is a perfectly reasonable concern and a common source of confusion for beginners.

The short answer:

- Yes, in divide-and-conquer algorithms, the heavy lifting is done by:
 - the **conquer** step (combining solutions), and
 - the **base cases** (which terminate recursion).

And yes, the intermediate steps in these algorithms can be interpreted as being “hollow.”

Who Does the Heavy Lifting in Divide and Conquer?

Example: Fibonacci

Naive (incorrect) recursion

```
int fibonacci(int n) {  
    return fibonacci(n-1) + fibonacci(n-2); // no base cases!  
}
```

Correct recursive definition

```
int fibonacci(int n) {  
    if (n == 0) return 0;           // base case  
    if (n == 1) return 1;           // base case  
    return fibonacci(n-1) + fibonacci(n-2); // conquer (combine)  
}
```

- The *conquer* step is the combination `fibonacci(n-1) + fibonacci(n-2)`.
- The *base cases* ensure termination; once reached, the recursion stack unwinds and the combinations propagate results upward.

Who Does the Heavy Lifting? — Merge Sort

- Heavy lifting:
 - **Base case:** size ≤ 1 (already sorted).
 - **Conquer step:** *merge* two sorted halves.

Recursive structure

MergeSort(A)

```
if  $|A| \leq 1$ : return A                                     (base case)
Split A into L and R (roughly equal sizes)
L'  $\leftarrow$  MergeSort(L)
R'  $\leftarrow$  MergeSort(R)
return Merge(L', R')                                     (conquer)
```

The Merge($\cdot, \cdot$) function is shown on the next slide.

Who Does the Heavy Lifting? — Merge Sort

Merge(L, R)

Initialize empty list S

While L and R are nonempty: append smaller front to S

Append any remaining items of L, then of R, to S

Return S

CSCI3160 Design and Analysis of Algorithms (2025 Fall)

Divide and Conquer: Fast Fourier Transform

Instructor: Xiao Liang

Department of Computer Science and Engineering
Chinese University of Hong Kong

Motivation: Multiply Polynomials Faster

We have seen divide-and-conquer speeding up many tasks. Next target: polynomials.

Example:

$$(1 + 2x + 3x^2) \cdot (2 + x + 4x^2) = 2 + 5x + 12x^2 + 11x^3 + 12x^4.$$

More generally, for degree- d polynomials

$$A(x) = \sum_{i=0}^d a_i x^i, \quad B(x) = \sum_{i=0}^d b_i x^i,$$

their product $C(x) = A(x)B(x) = \sum_{k=0}^{2d} c_k x^k$ with

$$c_k = \sum_{i=0}^k a_i b_{k-i} \quad (\text{treat } a_i, b_i = 0 \text{ if } i > d).$$

Naive time: $\Theta(d^2)$. Can we do better?

Key Idea: Two Representations of a Polynomial

Two equivalent representations of $A(x)$:

- ① Coefficients: $(a_0, a_1, \dots, a_d)$
- ② Values at $d + 1$ (arbitrary) distinct points (aka point-value pairs):

$$(x_0, A(x_0)), (x_1, A(x_1)), \dots, (x_d, A(x_d))$$

The above is due to a simply fact for polynomials:

Fact: A degree- d polynomial is uniquely determined by its values at **any** $d + 1$ **distinct** points.

Think: How to prove it? (Proof: view coefficients as the variable you want to solve in a linear equation system.)

Why point-value representation?

Why are we interested in point-value representation?

- If $C(x) = A(x)B(x)$ has degree $2d$, then for any point z ,

$$C(z) = A(z) \cdot B(z).$$

Thus in the *value* representation, multiplication is linear time, i.e., just $(2d + 1) = O(d)$ products.

Evaluate–Multiply–Interpolate Paradigm

The discussion so far inspires the following idea for polynomial multiplication:

Input: coefficients of $A(x)$, $B(x)$ of degree d .

- 1 Selection: pick n distinct points $x_0, \dots, x_{n-1}$, where $n = 2d + 1$.
- 2 **Evaluation:** compute $A(x_k)$ and $B(x_k)$ for all $k \in \{0, 1, \dots, n-1\}$.
- 3 Pointwise multiply: $C(x_k) = A(x_k)B(x_k)$ for all $k \in \{0, 1, \dots, n-1\}$.
- 4 **Interpolation:** recover $C(x)$ from $\{(x_k, C(x_k))\}_{k \in \{0, 1, \dots, n-1\}}$.

Bottlenecks: how to perform fast **Evaluation** and **Interpolation**?

Baseline vs. Goal

Evaluating a degree- n polynomial at one point costs at least $O(n)$ time¹. At n points, the baseline is $\Theta(n^2)$.

The key intuition behind Fast Fourier Transform: if our goal is to represent a degree- n polynomial by its values at n points, we are free to choose which points to use. So is it possible to pick a “highly structured” set of points that makes evaluation cheaper? By choosing points with special algebraic structure, we might be able to exploit these structures to reduce the total cost from the naive $\Theta(n^2)$.

Spoiler: Yes, we can. FFT successfully implements the above intuition using the set of n -th roots of unity as the evaluation points, reducing the total cost from $\Theta(n^2)$ to $\Theta(n \log n)$.

¹In fact, achieving $O(n)$ is non-trivial. We need to use Horner's method.

Even-Odd Split

Let us decompose a given polynomial in a special manner: (assume for convenience that n is even):

$$\begin{aligned} A(x) &= a_0 + a_1x + a_2x^2 + a_3x^3 + \dots + a_nx^n \\ &= (a_0 + a_2x^2 + a_4x^4 + \dots + a_nx^n) + (a_1x + a_3x^3 + a_5x^5 + \dots + a_{n-1}x^{n-1}) \\ &= (a_0 + a_2x^2 + a_4x^4 + \dots + a_nx^n) + x \cdot (a_1 + a_3x^2 + a_5x^4 + \dots + a_{n-1}x^{n-2}) \\ &= A_{\text{even}}(x^2) + x \cdot A_{\text{odd}}(x^2), \end{aligned}$$

where

$$\begin{aligned} A_{\text{even}}(x) &= a_0 + a_2x^1 + a_4x^2 + \dots + a_nx^{n/2} \\ A_{\text{odd}}(x) &= a_1 + a_3x^1 + a_5x^2 + \dots + a_{n-1}x^{n/2-1} \end{aligned}$$

Observation: that $A_{\text{even}}(x)$ collects all even coefficients of $A(x)$, and A_{odd} collects all odd coefficients of $A(x)$. Both of them a of degree $\leq n/2$.

Even-Odd Split: Example

Let's see a concrete example:

$$\begin{aligned}A(x) &= 3 + 4x + 6x^2 + 2x^3 + x^4 + 10x^5 \\&= (3 + 6x^2 + x^4) + (4x + 2x^3 + 10x^5) \\&= (3 + 6x^2 + x^4) + x \cdot (4 + 2x^2 + 10x^4) \\&= A_{\text{even}}(x^2) + x \cdot A_{\text{odd}}(x^2),\end{aligned}$$

where

$$\begin{aligned}A_{\text{even}}(x) &= 3 + 6x + x^2 \\A_{\text{odd}}(x) &= 4 + 2x + 10x^2\end{aligned}$$

Divide-and-Conquer via Even–Odd Split

Split $A(x)$ into even and odd powers:

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2),$$

where A_{even} collects even-indexed coefficients and A_{odd} collects odd-indexed coefficients.

If we evaluate at paired points $\pm x_i$, computations overlap:

$$A(x_i) = A_{\text{even}}(x_i^2) + x_i A_{\text{odd}}(x_i^2), \quad A(-x_i) = A_{\text{even}}(x_i^2) - x_i A_{\text{odd}}(x_i^2).$$

Thus, n evaluations at $\{\pm x_0, \dots, \pm x_{n/2-1}\}$ reduce to evaluations of A_{even} and A_{odd} at $\{x_0^2, \dots, x_{n/2-1}^2\}$ (two subproblems of size $n/2$) plus linear-time combination.

Recurrence and the Obstacle (1/2)

So far, we've seen that if we choose a special set $S = \{\pm x_0, \pm x_1, \dots, \pm x_{n/2-1}\}$, and:

- **Original problem:** evaluate $A(x)$ on all the points in S .

Note: You can assume that the actual degree of $A(x)$ is d . We may choose n to be any even number greater than $2d + 1$ —this is a redundant parameter choice for $A(x)$ itself. However, we will later need $2d + 1$ points to represent the product $C(x)$, so we must have $n \geq 2d + 1$. Also note that there is no harm in representing a degree- d polynomial with more than $d + 1$ points (here, n), even though $d + 1$ points already suffice.

This problem can be reduced to first solve the same problem for $S' = \{x_0^2, x_1^2, \dots, x_{n/2-1}^2\}$, i.e.,

- **Reduced problem:** evaluate $A(x)$ on all the points of S' .

Two important observations:

- 1 $|S'| = |S|/2$
- 2 Given the answer to the **reduced problem**, we can solve the **original problem** in $O(n)$ time. (Think: convince yourself this is true, using the equations in the previous slide)

Recurrence and the Obstacle (1/2)

If this reduction could recurse, we would get

$$T(n) = 2T(n/2) + O(n),$$

which solves to $T(n) = O(n \log n)$.

Obstacle: the plus-minus trick only works to reduce S to S' . We cannot reuse the idea to reduce S' further to a S'' of half size. So, the recursion got stuck at S' .

Solution: Let's utilize complex numbers.

Utilizing Complex Numbers (A Toy Example)

Set $S = \{1, -1, i, -i\}$, where i is the **Imaginary unit**, i.e., $i = \sqrt{-1}$.

This S supports two layer of recursion:

- $S' = \{1, -1\}$ (obtained by squaring each element in S)
- $S'' = \{1\}$ (obtained by squaring each element in S')

That is, if we want to evaluate a degree-3 polynomial $A(x)$ on every point of S , we can first do it at S' , which can further be reduced to doing the evaluation on S'' .

This implement the above recurse $T(n) = 2T(n/2) + O(n)$ for the special case of $n = 4$.

Important: Using a high-dimensional analog of i , we can generalize this idea to any n .

Recalling Complex Numbers: Fundamental Theorem of Algebra

Statement Every polynomial of degree $n \geq 1$ with complex coefficients has *exactly* n roots in $\mathbb{C}$, *counted with multiplicity*. Equivalently, for

$$p(z) = a_n z^n + a_{n-1} z^{n-1} + \cdots + a_1 z + a_0 \quad (a_n \neq 0),$$

there exist $\zeta_1, \dots, \zeta_n \in \mathbb{C}$ such that

$$p(z) = a_n \prod_{k=1}^n (z - \zeta_k),$$

where each root appears according to its multiplicity.

Fun Facts: This theorem was first established by Carl Friedrich Gauss in year 1799. During his life, Gauss offered not less than four different proofs to the theorem, covering a timespan of fifty years spanning his entire adult life.

Recalling Complex Numbers: Roots of Unity

Solutions to $x^2 = 1$?

- $x = 1$ and $x = -1$

Solutions to $x^3 = 1$?

- $x = 1$, $x = \cos(\frac{2\pi}{3}) + i \cdot \sin(\frac{2\pi}{3})$, and $x = \cos(\frac{4\pi}{3}) + i \cdot \sin(\frac{4\pi}{3})$

In general, there are n solutions to $x^n = 1$, they are:

- $x = \cos(\frac{2k\pi}{n}) + i \cdot \sin(\frac{2k\pi}{n})$, $\forall k \in \{0, 1, \dots, n-1\}$

They are called the n -th roots of unity

Recalling Complex Numbers: Roots of Unity

In general, there are n *distinct* solutions to $x^n = 1$, they are:

- $x = \cos(\frac{2k\pi}{n}) + i \cdot \sin(\frac{2k\pi}{n}), \forall k \in \{0, 1, \dots, n-1\}$

We denote $\omega_n = \cos(\frac{2\pi}{n}) + i \cdot \sin(\frac{2\pi}{n})$, then

$$\omega_n^0 = \cos(\frac{2 \cdot 0 \cdot \pi}{n}) + i \cdot \sin(\frac{2 \cdot 0 \cdot \pi}{n}) = 1$$

$$\omega_n^1 = \cos(\frac{2 \cdot 1 \cdot \pi}{n}) + i \cdot \sin(\frac{2 \cdot 1 \cdot \pi}{n})$$

$$\omega_n^2 = \cos(\frac{2 \cdot 2 \cdot \pi}{n}) + i \cdot \sin(\frac{2 \cdot 2 \cdot \pi}{n})$$

...

$$\omega_n^{n-1} = \cos(\frac{2 \cdot (n-1) \cdot \pi}{n}) + i \cdot \sin(\frac{2 \cdot (n-1) \cdot \pi}{n})$$

Therefore, it holds that $\omega_n^k = \cos(\frac{2k\pi}{n}) + i \cdot \sin(\frac{2k\pi}{n})$ for all $k \in \{0, 1, \dots, n-1\}$.

Thus, the n distinct n -th roots of unity (i.e., solutions to $x^n = 1$) can be equivalently written as:

$$\omega_n^0, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-2}, \omega_n^{n-1},$$

where $\omega_n = \cos(\frac{2\pi}{n}) + i \cdot \sin(\frac{2\pi}{n})$. This ω_n is sometimes referred to as the **principal**² n -th root of unity. (Note that there are n different n -th roots of unity. But there is only one **principal** n -th root of unity.)

Pictorially, the n -th roots of unity are equally spaced on the unit circle, starting from $\omega_n^0 = 1$, with an angle of $2\pi/n$ between adjacent ω_n^k and ω_n^{k+1} for all $k \in \{0, 1, \dots, n-1\}$.

[draw the picture on the whiteboard]

²We use the word “principal” to refer to the root of unity having smallest positive complex argument. But this is an abuse of the term. See [the discussion on Wolfram MathWorld](#).

Recalling Complex Numbers: Euler's Formula

This is also a good place to recall Euler's formula: for all real number θ ,

$$e^{i\cdot\theta} = \cos(\theta) + i \cdot \sin(\theta).$$

Now, let's memorize the following three formats of the n -th roots of unity: for all $k \in \{0, 1, \dots, n-1\}$:

$$\cos\left(\frac{2 \cdot k \cdot \pi}{n}\right) + i \cdot \sin\left(\frac{2 \cdot k \cdot \pi}{n}\right) = e^{\frac{2 \cdot k \cdot \pi}{n}} = \omega_n^k,$$

where recall that ω_n is the principal n -th root of unity.

Important Facts about Roots of Unity

The following facts will be crucial to the design of FFT:

- **Fact 1:** For all $n \in \mathbb{N}$, there are exactly n **distinct** n -th roots of unity, which we denote by

$$U_n = \{\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}\}.$$

- **Fact 2:** For all $n \in \mathbb{N}$ that is even, it holds that $\omega_n^{k+\frac{n}{2}} = -\omega_n^k$ for all $k \in \{0, 1, \dots, \frac{n}{2} - 1\}$.
- **Fact 3:** For all $n \in \mathbb{N}$ that is even, squaring each element of U_n will generate the set of $U_{\frac{n}{2}}$, i.e., the set of $\frac{n}{2}$ -th roots of unity. More explicitly, it holds that $\omega_n^{2k} = \omega_{n/2}^{k \bmod (n/2)}$

[We will not discuss the proofs of these three facts. You should be able to prove them yourself, as they involve only high-school-level complex numbers.]

FFT Pseudocode I

Input. Polynomial $A(x)$ of degree at most $n - 1$, where $n = 2^m$ for some $m \geq 0$.

Goal. Evaluate $A(x)$ on all points in $U_n = \{\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}\}$ (i.e., the n -th roots of unity)

Base case: If $n = 1$, then $\deg A(x) \leq 0$, so $A(x) = a_0$ for a constant a_0 . Output $[a_0]$.

[Time complexity: $O(1)$]

Recursive case: Assume $n \geq 2$.

- 1 Split $A(x)$ into even and odd parts: $A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$. [Time complexity: $O(n)$]
- 2 Make **two recursive FFT calls (of size $n/2$)** (Think: convince yourself this is true!) to evaluate both $A_{\text{even}}(y)$ and $A_{\text{odd}}(y)$ on all the points in $U_{n/2} = \{\omega_{n/2}^0, \omega_{n/2}^1, \dots, \omega_{n/2}^{n/2-1}\}$. In particular, this step yields the following values

$$A_{\text{even}}(\omega_{n/2}^0), A_{\text{even}}(\omega_{n/2}^1), \dots, A_{\text{even}}(\omega_{n/2}^{n/2-1})$$

$$A_{\text{odd}}(\omega_{n/2}^0), A_{\text{odd}}(\omega_{n/2}^1), \dots, A_{\text{odd}}(\omega_{n/2}^{n/2-1})$$

[Time complexity: $2f(n/2)$]

FFT Pseudocode II

- ③ $\forall k \in \{0, 1, \dots, n-1\}$, compute

$$A(\omega_n^k) = A_{\text{even}}(\omega_n^{2k}) + \omega_n^k A_{\text{odd}}(\omega_n^{2k}),$$

using the values of $A_{\text{even}}(y)$ and $A_{\text{odd}}(y)$ at $\omega_n^{2k} = \omega_{n/2}^{k \bmod (n/2)}$ (**this is Fact 3!**) computed in the previous step. [Time complexity: $O(n)$]

- ④ Output the values $(A(\omega_n^k))_{k=0}^{n-1}$ in order of increasing k .
[Time complexity: $O(n)$]

This finishes the description of the FFT algorithm.

Time complexity analysis:

$$f(n) = \begin{cases} O(1) & n = 1 \text{ (based case)} \\ 2 \cdot f(n/2) + O(n) & n \geq 2 \end{cases},$$

which solves to $f(n) = O(n \log n)$.

Interpolation as “Inverse FFT” I

The FFT algorithm, in $O(n \log n)$ time:

- **Input:** polynomial $A(x)$ in its coefficients representation:

$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^{n-1}.$$

- **Output:** pair-value representation of $A(x)$ at all the n -th roots of unity:

$$((\omega_n^0, A(\omega_n^0)), (\omega_n^1, A(\omega_n^1)), \dots, (\omega_n^{n-1}, A(\omega_n^{n-1})),)$$

Question: Can we convert the pair-value representation back to the coefficients representation (aka the task of “interpolation”) in $O(n \log n)$ time?

More formally:

Interpolation as “Inverse FFT” II

- **Input:** pair-value representation of $C(x)$ at all the n -th roots of unity:

$$((\omega_n^0, C(\omega_n^0)), (\omega_n^1, C(\omega_n^1)), \dots, (\omega_n^{n-1}, C(\omega_n^{n-1})),).$$

You are guaranteed that these points come from a polynomial $C(x)$ of degree at most n .

- **Input:** The unique polynomial $C(x)$ in its coefficients representation:

$$C(x) = c_0 + c_1x + c_2x^2 + \dots + c_nx^{n-1}.$$

Solution: This interpolation task can also be done in $O(n \log n)$ time, using an algorithm called “inverse FFT.” It can be roughly understood as “running FFT in the reverse order.” We won’t talk about the details during lecture time. Interested students could refer to Section 2.6.3 of [DPV]. [This part will not appear in quizzes/exams.]

Evaluate–Multiply–Interpolate Paradigm, Finalized

Recall the Evaluate–Multiply–Interpolate Paradigm we talked about earlier for polynomial multiplication. Now, we can complete it using FFT

Input: coefficients of $A(x), B(x)$ of degree d .

- 1 Let $U_n = \{\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}\}$ be the n -th roots of unity, where n is the smallest power of two that is $\geq 2d + 1$.
- 2 Evaluation: compute $A(\omega_n^k)$ and $B(\omega_n^k)$ for all $k \in \{0, 1, \dots, n-1\}$.
[using FFT, in time $O(n \log n)$]
- 3 Pointwise multiply: $C(\omega_n^k) = A(\omega_n^k)B(\omega_n^k)$ for all $k \in \{0, 1, \dots, n-1\}$.
[in time $O(n)$]
- 4 Interpolation: recover $C(x)$ from $\{(\omega_n^k, C(\omega_n^k))\}_{k \in \{0, 1, \dots, n-1\}}$.
[using “inverse FFT”, in time $O(n \log n)$]

Congratulations! You now know how to multiply two degree- n polynomials in time $O(n \log n)$.

A Note on Terminology

In this lecture, we use “FFT” to mean the algorithm that converts a polynomial from its coefficient representation to its point-value representation, and “inverse FFT” for the reverse procedure.

This terminology is not universal. While our usage is common (especially in the CS community), some authors—more often in mathematics—swap the names, calling our “FFT” the “inverse FFT,” and vice versa.

CSCI3160 Design and Analysis of Algorithms (2025 Fall)

Greedy 1: Activity Selection

Instructor: Xiao Liang¹

Department of Computer Science and Engineering
Chinese University of Hong Kong

¹These slides are primarily based on materials prepared by [Prof. Yufei Tao](#) (please refer to [Prof. Tao's version from 2024 Fall](#) for the original content). Some modifications have been made to better align with this year's teaching progress, incorporating student feedback, in-class interactions, and my own teaching style and research perspective.

In this lecture, we will commence our discussion of **greedy** algorithms, which enforce a simple strategy: make the **locally optimal** decision at each step. Although this strategy does not always guarantee finding a **globally optimal** solution, sometimes it does. The nontrivial part is to prove (or disprove) the global optimality.

Activity Selection — Real-World Example

Story: You're planning a one-day mini-conference. You have many candidate talks, each with a start time and an end time. You only have one room, so talks can't overlap.

Goal: Fit in as many talks as possible without time conflicts.

What does “overlap” mean? Two talks overlap if their time intervals share any time. Example:

- $[9:00, 10:00]$ and $[9:30, 10:30]$ overlap;
- $[9:00, 10:00]$ and $[10:00, 11:00]$ do *not* (if we allow back-to-back).

Your choices:

- You can include a talk or skip it.
- If you include it, you must avoid any other talk that overlaps with it.

Activity Selection: Formal Setup

Activity Selection

Input: A set S of n intervals of the form $[s, f]$ where s and f are integers.

Output: A subset T of disjoint intervals in S with the largest size $|T|$.

Remark: You can think of $[s, f]$ as the duration of an activity, and consider the problem as picking the largest number of activities that do not have time conflicts.

Activity Selection

Example: Suppose

$$S = \{[1, 9], [3, 7], [6, 20], [12, 19], [15, 17], [18, 22], [21, 24]\}.$$

$T = \{[3, 7], [15, 17], [18, 22]\}$ is an optimal solution, and so is $T = \{[1, 9], [12, 19], [21, 24]\}$.

A Greedy Algorithm for Activity Selection

Activity Selection

One-line summary: Pick the earliest-finishing interval, keep it, and discard all intervals that overlap it. Repeat until none remain.

Algorithm (formal)

Repeat until S becomes empty:

- Add to T the interval $\mathcal{I} \in S$ with the smallest finish time.
- Remove from S all the intervals intersecting $\mathcal{I}$ (including $\mathcal{I}$ itself)

Time Complexity: Convince yourself that this algorithm can be implemented in $O(n \log n)$ time.

Example: Suppose $S = \{[1, 9], [3, 7], [6, 20], [12, 19], [15, 17], [18, 22], [21, 24]\}$.

Let us rearrange the intervals in S in ascending order of finish time:
 $S = \{[3, 7], [1, 9], [15, 17], [12, 19], [6, 20], [18, 22], [21, 24]\}$.

We first add $[3, 7]$ to T , after which intervals $[1, 9]$ and $[6, 20]$ are removed. Now S becomes $\{[15, 17], [12, 19], [18, 22], [21, 24]\}$. The next interval added to T is $[15, 17]$, which shrinks S further to $\{[18, 22], [21, 24]\}$. After $[18, 22]$ is added to T , S becomes empty and the algorithm terminates.

Why the Greedy Choice?

Greedy Choice Property:

- Picking the earliest finishing activity leaves the most room for future choices.

Key Insight:

- If we always choose the earliest finishing compatible activity, we never miss out on a better solution.

Proof Outline: Exchange Argument

Note: there may be more than one optimal solution (think: can you construct an example for this fact?). Therefore, it is not accurate to say that our algorithm outputs “the” optimal solution. It is sufficient to show that our algorithm outputs an optimal solution.

Let:

- $G = \{g_1, g_2, \dots, g_k\}$ be the greedy solution
- $O = \{o_1, o_2, \dots, o_m\}$ be an optimal solution, ordered by increasing finish time.

Remarks:

- Each g_i (or o_i) is an interval!
- Since there is no overlap among the o_i 's, it holds for each i that $f_{o_{i-1}}$ is earlier than s_{o_i} , which is the start time of o_i .

Goal: Show that $k = m$ (greedy is optimal)

Some Simple Facts

Let:

- $G = \{g_1, g_2, \dots, g_k\}$ be the greedy solution
- $O = \{o_1, o_2, \dots, o_m\}$ be an optimal solution, ordered by increasing finish time.

Facts:

- ① Each g_i (or o_i) is an interval!
- ② Since there is no overlap among the o_i 's, it holds for each i that $f_{o_{i-1}}$ is earlier than s_{o_i} , which is the start time of o_i .
- ③ It must be that $k \leq m$; Otherwise, it violates the optimality of the optimal solution O .
- ④ So, to prove $k = m$ (our goal), it suffices to show that k is no less than m .

Exchange Argument in Action

Strategy: Transform O into G without reducing the number of activities using an **exchange argument**.

Assume:

- Greedy picks g_1 (earliest finishing)
- Optimal picks o_1 (maybe different)
- $f_{g_1} \leq f_{o_1}$ (where f_{g_1} denotes the finish time of activity g_1 . Similar for f_{o_1} .)

Exchange Step:

- Replace o_1 with g_1 in O
- Since g_1 finishes no later than o_1 , it's compatible with all following activities in O

Repeat the above steps to replace $o_2, o_3, \dots$

Finishing the Proof

If we repeat the above exchange argument, we can gradually update O in the following order:

$$O = \{o_1, o_2, o_3, \dots, o_m\}$$

$$O_1 = \{g_1, o_2, o_3, \dots, o_m\}$$

$$O_2 = \{g_1, g_2, o_3, \dots, o_m\}$$

$$\vdots$$

Assuming for the sake of contradiction that $k < m$, the following sequence would stop at

$$O_k = \{g_1, g_2, \dots, g_k, o_{k+1}, o_{k+2} \dots, o_m\}.$$

Finishing the Proof

Such an O_k contradicts the description of the greedy algorithm:

- Note that the finish time of g_k is earlier than the finish time of all intervals in

$$\{O_{k+1}, O_{k+2} \dots, O_m\},$$

and g_k do not overlap with any one of them.

- Given the above, by definition, the greedy algorithm would **not** stop at g_k , because there are still intervals satisfying the algorithm's condition to be added to the solution set G .
- **Contradiction:** However, by our assumption, g_k is the last element in G . This gives a contradiction.

Thus, our assumption that $k < m$ is incorrect. It must be that $k = m$.

This finishes the proof

CSCI3160 Design and Analysis of Algorithms (2025 Fall)

Greedy 2: Minimum Spanning Trees

Instructor: Xiao Liang¹

Department of Computer Science and Engineering
Chinese University of Hong Kong

¹These slides are primarily based on materials prepared by [Prof. Yufei Tao](#) (please refer to [Prof. Tao's version from 2024 Fall](#) for the original content). Some modifications have been made to better align with this year's teaching progress, incorporating student feedback, in-class interactions, and my own teaching style and research perspective.

Undirected Weighted Graphs

Let $G = (V, E)$ be an undirected graph. Let w be a function that maps each edge e of G to a positive integer value $w(e)$, which we call the **weight** of e .

An **undirected weighted graph** is defined as a pair (G, w) .

We will denote an edge between vertices u and v in G as (u, v) . Note that the ordering of u, v does not matter (i.e., G is undirected).

We consider that G is **connected**, namely, there is a path between **any** two vertices in V .

Motivation: Cheapest Network

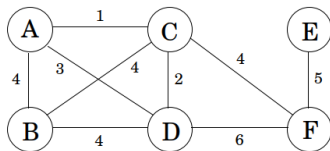


Figure: Picture taken from [Section 5.1, DPV]

Suppose you must link a collection of computers/sites (as shown above):

- Nodes are computers (or cities); edges are potential links.
- Each link has a maintenance cost (edge weight).
- Goal: connect everyone while minimizing total cost.

What structure should the optimal network have?

A Running Example

After some initial thoughts, we realize that the solution must be:

- **Connected:** to ensure all the sites are connected
- **Acyclic:** the optimal set of edges cannot contain a cycle, because removing an edge from this cycle would reduce the cost without compromising connectivity!

Undirected graphs of this kind are called **trees**. The particular tree we want is the one with minimum total weight, known as the **Minimum Spanning Tree**.

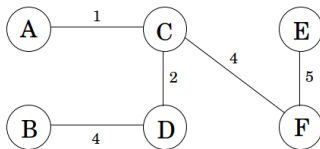


Figure: A MST for the Previous Graph

Formal Definition

Spanning Trees

Recall that a **tree** is defined as a connected undirected graph with no cycles.

Given a connected undirected weighted graph (G, w) with $G = (V, E)$, a **spanning tree** T is a tree satisfying the following conditions:

- The vertex set of T is V .
- Every edge of T is an edge in G .

That is, it is a tree (i.e., connected and contains no cycles) obtained by deleting some edges from G while ensuring that all the vertices of G are retained (i.e., do not become isolated).

The **cost** of T is defined as the sum of the weights of all the edges in T (note that T must have $|V| - 1$ edges).

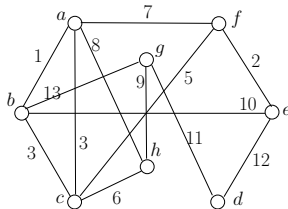
More Motivating Examples for MST

- Networking: Connect data centers or office sites with cables (links have costs), where the goal is to connect everyone at minimum total cost.
- Transportation: Build roads, pipelines, or power lines to reach all cities while minimizing construction cost.
- Clustering in ML: Use MST to reveal structure; cut the largest edges to form clusters (single-linkage).

An important shared pattern: We must connect all points while keeping total cost small.

A More Sophisticated Example

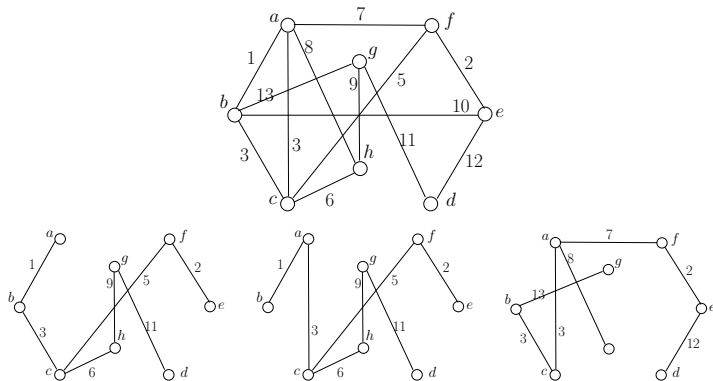
Example



The integer on each edge indicates its weight. For example, the weight of (g, h) is 9, and that of (d, g) is 11.

A More Sophisticated Example

Example



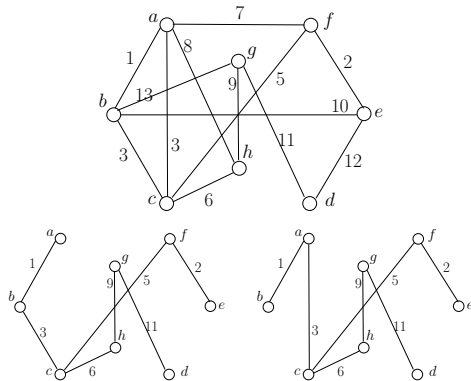
The second row shows three spanning trees (of the graph in the first row). The cost of the first two trees is 37, and that of the right tree is 48.

The Minimum Spanning Tree Problem

Given a connected undirected weighted graph (G, w) with $G = (V, E)$, the goal of the **minimum spanning tree (MST) problem** is to find a spanning tree of the smallest cost.

Such a tree is called an MST of (G, w) .

Example



Both trees in the second row are MSTs (of cost 37). This means that MSTs may not be unique.

Prim's Algorithm for MST

Choose an arbitrary vertex as your starting point (Think: why doesn't it matter where you start?)

The algorithm grows a tree T_{mst} by including one vertex at a time. At any moment, it divides the vertex set V into two parts:

- The set S of vertices that are already in T_{mst} .
- The set of other vertices: $V \setminus S$.

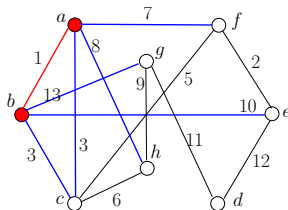
If an edge connects a vertex in S and a vertex in $V \setminus S$, we call it a **cross edge**.

Greedy: The algorithm works by repeatedly taking the **lightest** cross edge.

Example

Say, we start from vertex a . So, $S = \{a\}$ in the beginning.

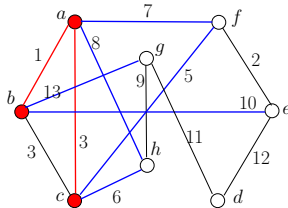
After the first iteration, $S = \{a, b\}$. The MST we are growing now contains the only red edge in the figure. Cross edges are shown in blue.



Cross edges (c, a) and (c, b) are both the lightest. Either one can be taken into the tree. Without loss of generality, suppose we take (c, a) .

Example

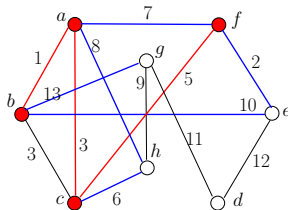
Now $S = \{a, b, c\}$. The MST we are growing now contains the red edges in the figure. Cross edges are shown in blue.



Note that (b, c) is not a cross edge.
The lightest cross edge is (c, f) .

Example

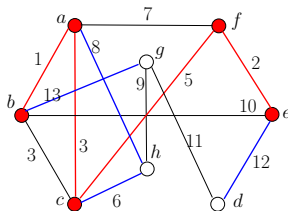
Now $S = \{a, b, c, f\}$. The MST we are growing now contains the red edges in the figure. Cross edges are shown in blue.



The lightest cross edge is (e, f) .

Example

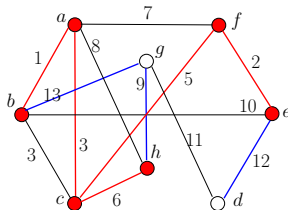
Now $S = \{a, b, c, f, e\}$. The MST we are growing now contains the red edges in the figure. Cross edges are shown in blue.



The lightest cross edge is (c, h) .

Example

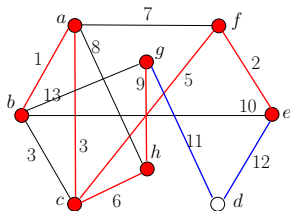
Now $S = \{a, b, c, f, e, h\}$. The MST we are growing now contains the red edges in the figure. Cross edges are shown in blue.



The lightest cross edge is (g, h) .

Example

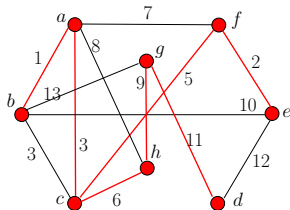
Now $S = \{a, b, c, f, e, h, g\}$. The MST we are growing now contains the red edges in the figure. Cross edges are shown in blue.



The lightest cross edge is (g, d) .

Example

Now $S = \{a, b, c, f, e, h, g, d\}$. We now have the final MST.



Running Time

Think: How to implement Prim's algorithm in $O((|V| + |E|) \cdot \log |V|)$ time? (We will discuss this in a tutorial.)

Proving the Correctness of Prim

We will prove correctness by induction:

- Maintain an invariant condition through the induction steps:
 - **Invariant Condition:** the edges selected so far by Prim are contained in **some** MST.
- At each induction step, we use an **exchange argument** to update the tree. Here, we will rely on three easy-yet-crucial graph-theoretic facts.
- At the end of the induction: the invariant condition implies that we have a spanning tree that is also minimal (aka a MST). (This step does not trivially follow from the invariant condition. It requires some extra efforts.)

In the following, we first state the three graph-theoretic facts, and then present the induction-based proof.

Graph Fact 1: Adding Edge to Tree Forms Cycle

Fact (Fact 1)

If $T = (E_T, V_T)$ is a tree and an edge $e \notin E_T$ connects two vertices in V_T , then the following graph

$$T' := (E_T \cup \{e\}, V_T)$$

contains exactly one cycle.

Proof.

- Since T is connected, there is already a unique path between the endpoints of e .
- Adding e creates a cycle by joining the ends of this path.
- As trees are acyclic, this must be the only cycle. [Think: how to formally prove this statement?]



Graph Fact 2: Removing Edge from Cycle Keeps Connectivity

Fact (Fact 2)

Let C be a set of edges that form a cycle in a connected graph. Removing any edge e from C keeps the graph connected.

Proof.

- Since the removed edge was part of a cycle, there remains an alternative path between its endpoints.
- Thus, all vertices remain reachable from one another.



Graph Fact 3

Fact (Fact 3)

Let $G = (V, E)$ be an undirected graph, and let T be a subgraph of G . Then the following are equivalent:

- ① T is a **spanning tree** of G *(Note that we didn't say "minimum" spanning tree here.)*
- ② T is **connected** and has exactly $|V| - 1$ edges

We prove this fact in the following two slides.

Direction 1: Spanning Tree $\Rightarrow$ Connectivity and $|V| - 1$ Edges

Assume: T is a spanning tree of G

Then by definition:

- T is connected
- T is acyclic
- T spans all vertices in V

Since any tree on $|V|$ vertices has exactly $|V| - 1$ edges, thus we know that T is connected and has exactly $|V| - 1$ edges.

Direction 2: Connectivity and $|V| - 1$ Edges $\Rightarrow$ Spanning Tree

Assume: T (being a subgraph of G) is connected and has exactly $|V| - 1$ edges.

- 1 First, note that T must be acyclic.

Proof: Otherwise, removing one edge from the cycle does not break connectivity; however, it is impossible to have a connected graph of $|V|$ vertices but with only $|V| - 2$ edges.

- 2 Also, T must span all vertices.

Proof: Otherwise, T can contain at most $|V| - 1$ vertices. However, recall that T has only $|V| - 1$ edges. These two conditions imply that T must contain a cycle, contradicting to the acyclic property we just established for T above.

In summary: T is connected, acyclic, and spans all vertices $\Rightarrow T$ is a spanning tree.

Starting the Proof of the Correctness of Prim

Summarizing the three facts:

- ① **Fact 1:** If $T = (E_T, V_T)$ is a tree and an edge $e \notin E_T$ connects two vertices in V_T , then the graph $T' := (E_T \cup \{e\}, V_T)$ contains exactly one cycle.
- ② **Fact 2:** Let C be a set of edges that form a cycle in a connected graph. Removing any edge e from C keeps the graph connected.
- ③ **Fact 3:** Let $G = (V, E)$ be an undirected graph, and let T be a subgraph of G . Then the following are equivalent:
 - T is a **spanning tree** of G
 - T is **connected** and has exactly $|V| - 1$ edges

With these three facts, we proceed to prove the correctness of Prim, using mathematical induction.

Inductive Proof Setup (for the Correctness of Prim)

Let:

- $G = (V, E)$: connected, undirected graph with edge weights.
- T : tree built by Prim's algorithm.
- T^* : any MST of G .
- E_k : first k edges selected by Prim.
- $S_k \subset V$: vertices included so far.

Through out the steps of the induction, we will maintain an **invariant condition**:

- in the k -th step, the tree formed by E_k is a sub-tree of some MST.

Base Case

- Initially, $E_0 = \emptyset$
- Any MST trivially contains E_0
- Invariant holds: E_0 is a subgraph of T^* .

[Some students are not comfortable with using the “corner case” E_0 as the induction basis. You could simply run the algorithm for one step and use E_1 as your induction basis.]

Inductive Hypothesis

Suppose after k steps:

- Prim has selected E_k
- **Invariant Condition** is true: There exists an MST T^* such that $E_k \subseteq T^*$

Next: Show that after adding edge e in the $k + 1$ -th step, the new set $E_{k+1} = E_k \cup \{e\}$ is also a sub-tree of some MST (that might be different from T^*).

Step $k + 1$: Two Cases

Let $e = (u, v)$ be the next edge chosen by Prim:

- $u \in S_k, v \notin S_k$
- e is the minimum-weight edge crossing the cut

There are two possibilities:

- **Case 1:** $e \in T^* \Rightarrow E_{k+1}$ is a sub-tree of T^*
- **Case 2:** $e \notin T^*$: We will use an exchange argument to construct another MST T' such that E_{k+1} is a sub-tree of T' .

Exchange Argument (Case 2)

Since T^* is a spanning tree:

- There is a unique path in T^* between u and v (recall that they are the two end points of the edge e)
- This path must contain some edge f crossing the cut $(S_k, V \setminus S_k)$

Construct:

$$T' := T^* + e - f$$

That is, T' is obtained by adding edge e to T^* , and then removing edge f from it.

It is obvious that $E_{k+1}(= E_k \cup \{e\})$ is a sub-graph of T' .

We next prove that T' is also a MST for G .

We first prove that T' is a spanning tree:

- By Fact 1: $T^* + e$ contains a cycle. (Also note that f is a part of this cycle.)
- By Fact 2: $T' (= T^* + e - f)$ is connected.
- Since T^* is a MST, it follows from by Fact 3 that T^* has exactly $|V| - 1$ edges.

This implies that T' must also has $|V| - 1$ edges (as it is obtained by $T' = T^* + e - f$).

The above shows that T' is connected and has exactly $|V| - 1$ edges. Then, by Fact 3, T' is a spanning tree.

We now prove that T' is MST:

- e is the lightest edge across the cut
- So $w(e) \leq w(f)$
- $T' (= T^* + e - f)$ has weight $\leq$ weight of T^*
- Therefore, T' is an MST

Termination

(Think: convince yourself that Prim's algorithm terminates after $|V| - 1$ steps)

After $|V| - 1$ steps:

- Prim has selected $|V| - 1$ (distinct) edges. Denote this set of edges by P .
- By induction, P must form a sub-tree of some MST. Denote this MST by T_{final} .
- By Fact 3, T_{final} must have exactly $|V| - 1$ edges.
- Thus, $P = T_{final}$.

$\Rightarrow$ Prim's algorithm outputs an MST (i.e., T_{final})